

C# BACKEND TRACK

Independent Assignment

Hospital Management System

Design and implement a console-based Hospital Management System in C# using the same architectural pattern you studied in the session.

Codeline Bootcamp · Backend Phase · Session 2 Assignment

01 DOMAIN OVERVIEW

You are building a **Hospital Management System (HMS)** — a console application that simulates the core operations of a real hospital. Hospitals deal with patients arriving for medical care, doctors who provide that care, scheduled appointments that connect a specific patient to a specific doctor, and medical records that capture what happened during each visit.

In a real hospital, a patient first gets registered in the system. Doctors are added to the system with their specialisation. Patients then book appointments with available doctors. After a visit takes place, a medical record is created documenting the diagnosis and prescribed medication. Appointments can be cancelled if a patient can no longer attend. Doctors can update their available time slots. The hospital administration needs to be able to view all of this data in various ways to manage day-to-day operations.

02 SYSTEM ENTITIES & CLASS MODELS

The system contains **five entities**. Each entity must be defined as a separate class inside a **Models** folder. There are **no navigation properties** — relationships between entities are expressed only by repeating the ID of one entity inside another entity, exactly as in the session code you studied.

Patient	
Property	Data Type
patientId	int
patientName	string
patientAge	int
patientGender	string
patientPhone	string
patientEmail	string
patientBloodType	string

Doctor	
Property	Data Type
doctorId	int
doctorName	string
doctorSpecialization	string
doctorPhone	string

Property	Data Type
doctorEmail	string
consultationFee	decimal

Appointment

Property	Data Type
appointmentId	int
patientId	int
doctorId	int
appointmentDate	string
appointmentTime	string
status	string

MedicalRecord

Property	Data Type
recordId	int
patientId	int
doctorId	int
appointmentId	int
diagnosis	string
prescription	string
visitDate	string
visitFee	decimal

AvailableSlot

Property	Data Type
slotId	int
doctorId	int
slotDate	string
slotTime	string
isBooked	bool

03 SYSTEM CONTEXT (IN-MEMORY STORAGE)

You must create a single **HospitalContext** class that acts as the in-memory storage for the entire system, identical in concept to the **EcommerceContext** you saw in the session. It holds one **List<T>** for each entity. In **Main()** you create one instance of this context, initialise all five lists as empty collections, and pass it to every function you implement.

04 REAL-LIFE SCENARIOS

The following scenarios describe real-life situations that happen in a hospital every day. Each scenario maps to a function you must implement inside **Program.cs**. You are responsible for deciding everything: the function signature, what to read from the user, what to validate, what to update in the context, and what to display. Study the scenario carefully — that is all the guidance you will receive.

Important: No hints, no step-by-step guides, and no solution code will be provided. Read each scenario as a real-life situation and reason about what the software must do. Every decision — inputs, outputs, validations, calculations, and data updates — is entirely yours to make.

Easy — Straightforward CRUD

Easy

01 Patient Registration

A new patient walks into the hospital reception desk for the very first time. The receptionist opens the system and enters the patient's personal information so that the hospital has a record of them. After the registration is complete, the system should confirm that the patient has been added and tell the receptionist the ID that was assigned to this patient.

Easy

02 Add a New Doctor

The hospital's HR department has hired a new doctor and wants to register them in the system so that patients can start booking appointments with them. The HR officer enters all of the doctor's professional details. Once the doctor is saved, the system confirms the addition and shows the new doctor's assigned ID.

Easy

03 View All Patients

A nurse needs to see a list of every patient currently registered in the hospital system. She opens the relevant menu option and the system displays all patients with their key details. If no patients have been registered yet, the system should let her know instead of displaying an empty screen.

Easy

04 View All Doctors by Specialization

A patient calls reception asking for a list of all cardiologists available at the hospital. The receptionist opens the system, selects the option to browse doctors, and types in the specialization they are looking for. The system displays only the doctors who match that specialization. If none are found, the system should say so clearly.

Medium — Logic & Cross-Entity Operations

Medium

05 Add an Available Time Slot for a Doctor

A doctor's secretary calls the system administrator to add new appointment slots for her doctor next week. The administrator opens the system, selects the doctor, and enters the date and time of each available slot. That slot is saved and is initially open for patients to book. The system confirms the slot has been added.

Medium

06 Book an Appointment

A patient calls the hospital and asks to book an appointment with a specific doctor. The receptionist opens the booking screen, selects the patient, then shows all available (unbooked) time slots for that doctor. The patient picks a slot, and the system records the appointment and marks that time slot as no longer available. If the doctor has no available slots, the system must inform the receptionist.

Medium

07 Cancel an Appointment

A patient phones the hospital and says they cannot make their appointment tomorrow. The receptionist looks up the appointment by its ID. The system updates the appointment status to show it is cancelled and makes the corresponding time slot available again so another patient can book it. If the appointment ID does not exist or the appointment is already cancelled, the system should respond accordingly.

Medium

08 Create a Medical Record After a Visit

A doctor has just finished seeing a patient. A nurse opens the system and creates a medical record for that visit. She links it to the relevant appointment, enters the doctor's diagnosis and the prescribed medication, and records the consultation fee that applies for this doctor. The system saves the record and confirms it has been created. The appointment's status should also be updated to reflect that the visit has been completed.

Hard — Aggregation & Multi-Step Business Logic

Hard

09 Generate a Patient Medical History Report

A doctor asks to see the complete medical history of a patient before starting a new consultation. The system asks for the patient's ID, then displays every medical record ever created for that patient — including the date of each visit, the name of the doctor who attended them, the diagnosis given, and the medication prescribed. At the end of the report the system also shows the total amount the patient has been charged across all their visits combined. If the patient has no records yet, the system should say so.

Hard

10 Doctor Workload and Revenue Summary

The hospital director opens the management dashboard and requests a full performance summary for all doctors. For each doctor, the system calculates and displays: the total number of completed appointments, the total revenue generated from consultation fees across all completed visits, and the number of appointments that were cancelled. The summary is sorted so that the doctor who generated the highest revenue appears first. If no appointments have been recorded yet for any doctor, the system informs the director.

05 PROJECT STRUCTURE & SUBMISSION

Folder / File	Purpose
Models/Patient.cs	Patient entity class
Models/Doctor.cs	Doctor entity class
Models/Appointment.cs	Appointment entity class
Models/MedicalRecord.cs	MedicalRecord entity class
Models/AvailableSlot.cs	AvailableSlot entity class
HospitalContext.cs	In-memory storage class (all 5 lists)
Program.cs	All functions + Main() menu loop

Submit your project as a zipped folder named **HMS_YourName.zip** via the course GitHub repository. Your project must compile and run without errors. Each scenario should be accessible from the Main() menu loop.